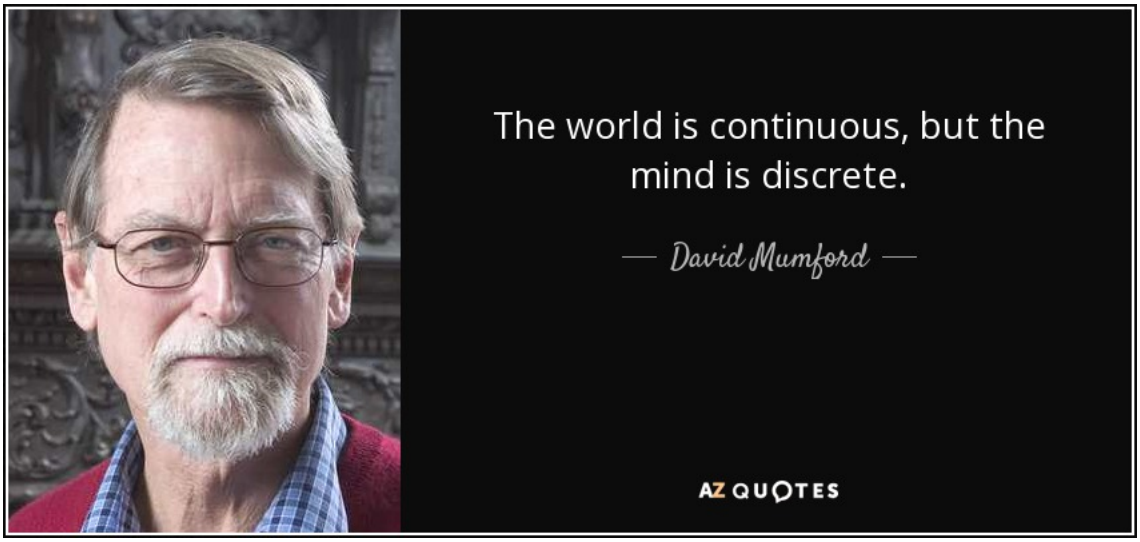
Logic-II

In this lecture we will learn about:

- Nested Quantifiers
- Satisfiability
- Normal Forms



Student Name: _____

Student ID: _____

Semester: _____

Welcome, to lecture-2 of discrete mathematics(M4), We revisited logic-it's history and importance through **linguistic plays of Lewis Carroll**.
We have revised and learned.

Self Reflection

"Did you understand the following?"

a) How to identify Propositions?

☐ Yes ☐ No

b) How to combine Propositions?

☐ Yes ☐ No

c) How to identify Predicate?

☐ Yes ☐ No

d) How to read/write Predicates?

☐ Yes ☐ No

e) How to quantify the truth of Predicates with $\forall$ and $\exists$.

☐ Yes ☐ No

1 Going more deeper into logic of quantified statement.

"An easy programming Challenge."

a) Write a program to find the following sum

$$Sum = \sum_{i=1}^{n} \sum_{j=1}^{n} (i + j).$$

Ex. *Write your "Python Code" below*

Just as we can place one loop inside another in a program, we can place one quantified statement **"inside"** another in predicate logic.

Statements become interesting when we involve more than one quantifier!

Example (Think Carefully): During every semester, there is a computer science class that **every student on campus** can take.

Ex. *Try to translate the above into logic*

Computing Example

Given an array A of size n, you have to check whether "Every element of an array $A[1, 2, ...n]$ appears at least twice."

For Example:

$$A = [3, 2, 1, 1, 4, 4, 2, 3, 1]$$

Ex. *Write one more example*

Logic

Let's translate the condition to be satisfied in logic using quantifiers.

Ex. *Write your answer below*

Computation

Implement in Python

Ex. *Write your answer below*

Recall Existential Quantifier

The predicate `sumOfTwoPrimes(n)` means:

"n can be written as the sum of two prime numbers."

Complete the logical structure below:

$$\text{sumOfTwoPrimes}(n) = \exists \underline{\quad\quad}, \underline{\quad\quad} \in \underline{\quad\quad\quad\quad} : \underline{\quad\quad\quad\quad\quad\quad\quad\quad}$$

Using, The predicate `sumOfTwoPrimes(n)`. Translate the "Goldbach's Conjecture".

" Every even integer greater than 2 can be written as sum of two primes"

Ex. *Write your answer below*

Important Note

Just as with nested loops in programs, the deeper the nesting of quantifiers, the harder an expression becomes for a reader to follow.

Extremely important tip for SOFTWARE DESIGN

Using well-chosen predicates (like *isPrime*, for example, in Refer Logic Sheet–1) in a logical statement can make it much easier to read, just like well-chosen (well-named / well-designed) functions make your software easier to read.

Pause and Think: Are you making the logic simpler for the reader, or just more compact?

2 Order of Quantification

Scenario

Consider the grid shown above.

Each cell of the grid contains a **switch**. When a switch is pressed, the **light in that cell blinks**.

Rows of the grid are indexed by r and columns by c.

Pressing switches in different cells may produce different outcomes:

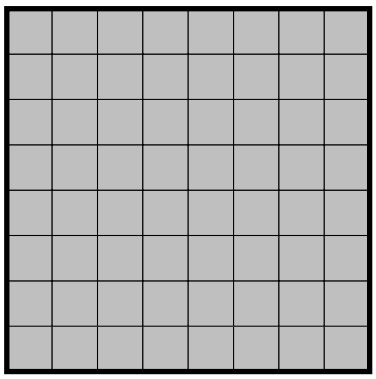
- Some switches may always make the light blink.
- Some switches may never make the light blink.

- Some rows may have many working switches, while others may have only one.

Our goal is to describe, using logic, **how the blinking behavior depends on rows and columns of the grid**, and how the *order of quantifying over rows and columns* changes the meaning of our statement.

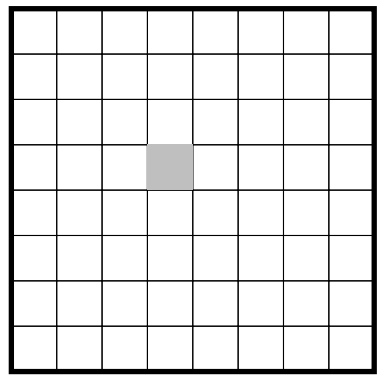
- ***For Example***, The **grid(a)** is completely filled and represents a known blinking pattern logically.
- For other grids **(b), (c), (d), (e), (f)** where lights are already blinking, **write the correct logical statement** involving $P(r, c)$ that describes the blinking pattern.

$\forall r : \forall c : \; P(r, c)$,
or, equivalently,
$\forall c : \forall r : \; P(r, c)$

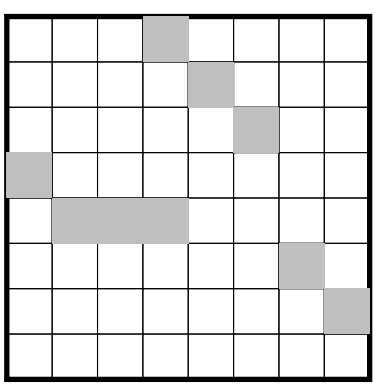


(a)

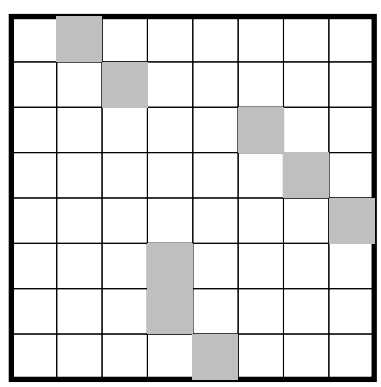
or, equivalently,



(b)

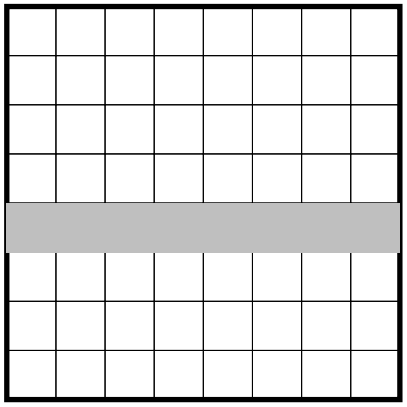


(c)

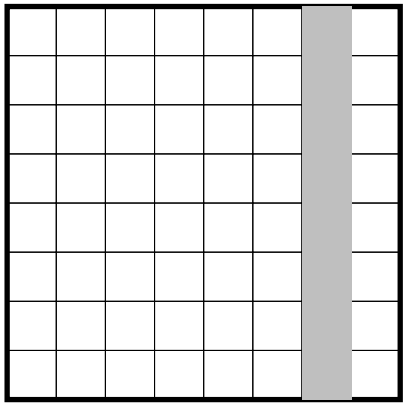


(d)

_____ _____



(e)



(f)

An illustration of order of quantification. Let r index a row of the grid, and let c index a column. If $P(r, c)$ is _____ in each filled cell, then the corresponding proposition is _____.

Commutativity of $\forall$ and commutativity of $\exists$

The order of quantifiers does matter when universal and existential quantifiers both appear in a proposition, the order of consecutive universal quantifiers doesn't matter. Neither does the order of consecutive existential quantifiers. That is, these quantifiers are *commutative*. Thus the following statements are theorems of predicate logic:

$$\forall x \in S : \forall y \in T : P(x, y) \iff \underline{\hspace{6cm}}$$

$$\exists x \in S : \exists y \in T : P(x, y) \iff \underline{\hspace{6cm}}$$

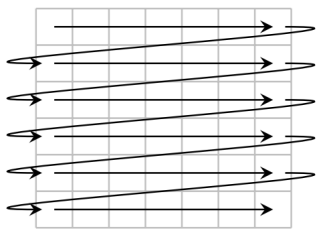
Nested quantification and nested loops

Nested quantifiers are like nested loops in a program: each quantifier acts as a loop. Changing the order of quantifiers, just like changing loop order, can change the meaning of the statement.
How thinking about the nested-loop analogy for nested quantifiers can be helpful. Imagine writing a nested loop to examine every element of a two-dimensional array. As long as iterations don't depend on each other, it doesn't matter whether we proceed through the array in row-major order (from top to bottom, going from left to right within each row) or column-major order (from left to right, going from top to bottom within each column).

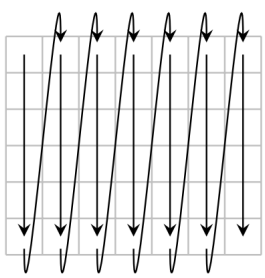
Logic

Going through a matrix in row-major order or column-major order. If every cell in the grid is inspected, then the final result is _____.

a)



b)



Computation

Implement in Python

a)

```
for _____ to m :
    for _____ to n :
        if _____ then
            return True
return False
```

b)

```
for _____ to n :
    for _____ to m :
        if A[i, j] then
            return True
return False
```

Negating Nested Quantifiers

Negating universal quantifiers.

Let's prove the equivalence you're using to debunk your partner's claim:

$$\neg\big[\forall x \in S : P(x)\big] \iff \underline{\qquad\qquad\qquad}. \tag{*}$$

It's probably easiest to view $(*)$ as a quantified version of the tautology

$$\neg(p \wedge q) \iff \underline{\qquad\qquad}$$

which was one of De Morgan's Laws from propositional logic. Here's the (oversimplified) intuition:

$$\begin{aligned}\neg\big[\forall x \in S : P(x)\big] &\approx \underline{\qquad\qquad\qquad\qquad\qquad\qquad}\\ &\equiv \underline{\qquad\qquad\qquad\qquad\qquad\qquad}\\ &\approx \underline{\qquad\qquad\qquad\qquad\qquad\qquad}.\end{aligned}$$

If we imagine the universe

$$S = \{x_1, x_2, \ldots, x_n\},$$

then

$$\forall x \in S : P(x)$$

means

$$\underline{\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad},$$

by De Morgan's Laws (propositional version), and

$$\exists x \in S : Q(x)$$

means

$$\underline{\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad}.$$

There is something slightly more subtle about $(*)$ because S might be infinite, but the idea is the same:

- *If there's an $a \in S$ such that $P(a) = False$, then*

  $$\exists x \in S : \underline{\qquad\qquad}$$

  is true (because a is an example) and

  $$\forall x \in S : \underline{\qquad\qquad}$$

  is false (because a is a counterexample).

- *On the other hand, if $P(a) = True$ for every $a \in S$, then*

  $$\exists x \in S : \underline{\qquad\qquad}$$

  is false and

  $$\forall x \in S : \underline{\qquad\qquad}$$

  is true.

Negating existential quantifiers.

Let's prove that this claim is a theorem, too:

$$\neg\big[\exists x \in S : P(x)\big] \iff \underline{\hspace{4cm}}. \tag{$\dagger$}$$

To see that ($\dagger$) is true for an arbitrary predicate P, we start with claim ($*$), but using the predicate

$$Q(x) = \underline{\hspace{2cm}}.$$

(Note that Q is also a predicate—so Negating universal quantifiers holds for Q too!) Thus we know that

$$\neg\big[\forall x \in S : Q(x)\big] \iff \underline{\hspace{4cm}},$$

and, because $p \Leftrightarrow q \equiv \neg p \Leftrightarrow \neg q$, we therefore also know that

$$\big[\forall x \in S : Q(x)\big] \iff \underline{\hspace{5cm}}.$$

But $Q(x)$ is just $\neg P(x)$ and $\neg Q(x)$ is just $P(x)$, by definition of Q, and so we have

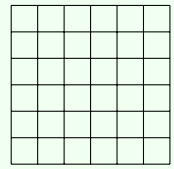
$$\big[\forall x \in S : \neg P(x)\big] \iff \underline{\hspace{5cm}}.$$

Thus we've now shown that ($\dagger$) is true for any predicate P, so it is a theorem.

Informally, these theorems say that "*everybody is P is false*" is equivalent to "*somebody isn't P*"; and, similarly, "*somebody is P is false*" is equivalent to "*everybody isn't P*".

No cell is filled $\equiv$ every cell is empty.

What does it mean for the statement $\exists r : \exists c : P(r, c)$ to be *false*? From the visual perspective in Figure (Refer Order of Quantification: Scenario), the statement is true if any cell in the grid is filled. There are two different ways of thinking about what it means for $\exists r : \exists c : P(r, c)$ to be false, then:



(1) *there does not exist a filled cell* _____

(2) *every cell is unfilled.* _____

(These two sentences probably don't sound very different to you, precisely because you probably already have some sense of what it means to negate quantifiers.) To write this equivalence formally, we repeatedly apply the rule for negating an existential quantifier:

$$\begin{aligned} & \underline{\qquad\qquad\qquad\qquad} && \text{“There does not exist a filled cell.”} \\ \equiv\ & \forall r : \underline{\qquad\qquad\qquad\qquad} \\ \equiv\ & \forall r : \underline{\qquad\qquad\qquad\qquad}. && \text{“Every cell is unfilled.”} \end{aligned}$$

Similarly, let's rewrite $\neg\,[\exists r : \forall c : P(r, c)]$:

$$\begin{aligned} & \underline{\qquad\qquad\qquad\qquad} && \text{“It's not the case that there's a row with all columns filled.”} \\ \equiv\ & \forall r : \underline{\qquad\qquad\qquad\qquad} \\ \equiv\ & \forall r : \underline{\qquad\qquad\qquad\qquad}. && \text{“Every row has at least one unfilled column.”} \end{aligned}$$

Triple negations

For a ternary predicate P, we have:

$$\begin{aligned} \neg\exists x : \forall y : \exists z : P(x, y, z) &\equiv \underline{\qquad\qquad\qquad\qquad} \\ &\equiv \underline{\qquad\qquad\qquad\qquad} \\ &\equiv \underline{\qquad\qquad\qquad\qquad}. \end{aligned}$$

Let's go back to propositional logic and talk about some extensions of logic.

Do you remember?

a) What is a **tautology**?

Ex. *Write your answer below*

b) What is a **contradiction**?

Ex. *Write your answer below*

3 Satisfiable and unsatisfiable propositions

Contradiction of $p \Leftrightarrow q$ ***and*** $p \oplus q$

We will investigate whether the compound proposition

$$(p \Leftrightarrow q) \ \wedge \ (p \oplus q)$$

can ever be true.

Step 1: Recall the meanings

- $p \Leftrightarrow q$ is true when _____.
- $p \oplus q$ is true when _____.

Step 2: Complete the truth table
Fill in each column carefully.

p	q	$p \Leftrightarrow q$	$p \oplus q$	$(p \Leftrightarrow q) \wedge (p \oplus q)$
T	T	_____	_____	_____
T	F	_____	_____	_____
F	T	_____	_____	_____
F	F	_____	_____	_____

Step 3: Observe the final column

- How many rows evaluate to **True**? _____
- What does this tell you about the satisfiability of the expression?

  _____

Conclusion (Write in your own words).
The proposition $(p \Leftrightarrow q) \wedge (p \oplus q)$ is a _____ because _____.

Key Insight. Although $p \Leftrightarrow q$ requires p and q to have the _____ truth value, $p \oplus q$ requires them to have _____ truth values.

Demanding satisfaction: $p \vee q \Rightarrow \neg p \wedge \neg q$.

Is the proposition $p \vee q \Rightarrow \neg p \wedge \neg q$ satisfiable?

Ex. *Write your answer below*

Satisfiable propositions,

A proposition is called **satisfiable** if it is *true under at least one truth assignment*.

Think & Write: Give one example of a satisfiable proposition.

Ex. *Write your answer below*

Unsatisfiable propositions / contradictions

A proposition is **unsatisfiable** if it is *not satisfiable*. Such a proposition is also called a **contradiction**.

Think & Write: Give one example of an unsatisfiable (contradictory) proposition.

Ex. *Write your answer below*

Taking it further

While satisfiability seems like a pretty precise technical definition that wouldn't matter all that much, the *satisfiability problem*—given a proposition φ, determine whether φ is satisfiable—turns out to be at the heart of the biggest open question in computer science today. If you figure out how to solve the satisfiability problem efficiently (or prove that it's impossible to solve efficiently), then you'll be the most famous computer scientist of the century.

Let's delve more into "Satisfiability"

a) Determine whether each of the compound propositions

$$(p \vee \neg q) \wedge (q \vee \neg r) \wedge (r \vee \neg p), \quad (p \vee q \vee r) \wedge (\neg p \vee \neg q \vee \neg r),$$

and

$$(p \vee \neg q) \wedge (q \vee \neg r) \wedge (r \vee \neg p) \wedge (p \vee q \vee r) \wedge (\neg p \vee \neg q \vee \neg r)$$

is satisfiable.

Ex. *Write your answer below*

4 Application of Satisfiability

Many problems, in diverse areas such as robotics, software testing, computer-aided design, machine vision, integrated circuit design, computer networking, and genetics, can be modeled in terms of propositional satisfiability.
we will see **how to use propositional satisfiability to model Sudoku puzzles**.
Sudoku A Sudoku puzzle is represented by a 9×9 grid made up of nine 3×3 subgrids, known as *blocks*, as shown in Figure 1. For each puzzle, some of the 81 cells, called *givens*, are assigned one of the numbers $1, 2, \ldots, 9$, and the other cells are blank. The puzzle is solved by assigning a number to each blank cell so that every row, every column, and every one of the nine 3×3 blocks contains each of the nine possible numbers. Note that instead of using a 9×9 grid, Sudoku puzzles can be based on $n^2 \times n^2$ grids, for any positive integer n, with the $n^2 \times n^2$ grid made up of n^2 $n \times n$ subgrids.

4.1 Try to fill it!

2			7				4	
4				2	3		7	9
1		7	4		9		2	
	9		3	7		4	6	
7	1	4		9			3	2
3		6	8	4			9	
	4	1		8	7			3
9			5		6	2	1	4
			9				8	

How to Encode?

To encode a Sudoku puzzle, let $p(i, j, n)$ denote the proposition that is true when the number n is in the cell in the ith row and jth column. There are $9 \times 9 \times 9 = 729$ such propositions, as i, j, and n all range from 1 to 9. For example, for the puzzle in Figure 1, the number 6 is given as the value in the fifth row and first column. Hence, we see that $p(5, 1, 6)$ is true, but $p(5, j, 6)$ is false for $j = 2, 3, \ldots, 9$.

Given a particular Sudoku puzzle, we begin by encoding each of the given values. Then, we construct compound propositions that assert that every row contains every number, every column contains every number, every 3×3 block contains every number, and each cell contains no more than one number. It follows, as the reader should verify, that the Sudoku puzzle is solved by finding an assignment of truth values to the 729 propositions $p(i, j, n)$ with i, j, and n each ranging from 1 to 9 that makes the conjunction of all these compound propositions true.

- For each cell with a given value, we assert $p(i, j, n)$ when the cell in row i and column j has the given value n.

- **Given.** We assert that every row contains every number:

  Ex. *Write your answer below*

- **Given.** We assert that every column contains every number:

Ex. *Write your answer below*

- **Given.** We assert that each of the nine 3×3 blocks contains every number:

  Ex. *Write your answer below*

- **Given.** To assert that no cell contains more than one number, we take the conjunction over all values of n, n', i, and j where each variable ranges from 1 to 9 and $n \neq n'$:

  Ex. *Write your answer below*

5 Solving Satisfiability Problems

Solving Satisfiability Problems

A truth table can be used to determine whether a compound proposition is satisfiable, or equivalently, whether its negation is a tautology. This can be done by hand for a compound proposition with a small number of variables, but when the number of variables grows, this becomes impractical. For instance, there are $2^{20} = 1{,}048{,}576$ rows in the truth table for a compound proposition with 20 variables. Clearly, you need a computer to help you determine, in this way, whether a compound proposition in 20 variables is satisfiable.

When many applications are modeled, questions concerning the satisfiability of compound propositions with hundreds, thousands, or millions of variables arise. Note, for example, that when there are 1000 variables, checking every one of the 2^{1000} (a number with more than 300 decimal digits) possible combinations of truth values of the variables in a compound proposition cannot be done by a computer in even trillions of years. No procedure is known that a computer can follow to determine in a reasonable amount of time whether an arbitrary compound proposition in such a large number of variables is satisfiable.

However, progress has been made developing methods for solving the satisfiability problem for the particular types of compound propositions that arise in practical applications, such as for the solution of Sudoku puzzles. Many computer programs have been developed for solving satisfiability problems which have practical use. Further, we will see the important role the propositional satisfiability problem plays in the study of the complexity of algorithms.